CRYPTOVAMPIRE: Automated Reasoning for the Complete Symbolic Attacker Cryptographic Model

Simon Jeanteur*, Laura Kovács*, Matteo Maffei*† and Michael Rawson*
*TU Wien, {firstname}.{lastname}@tuwien.ac.at
†Christian Doppler Laboratory Blockchain Technologies for the Internet of Things

***Abstract*—Cryptographic protocols are hard to design and prove correct, as witnessed by the ever-growing list of attacks even on protocol standards. *Symbolic models of cryptography* enable automated formal security proofs of such protocols against an idealized cryptographic model, which abstracts away from the algebraic properties of cryptographic schemes and thus misses attacks. *Computational models of cryptography* yield rigorous guarantees but support at present only interactive proofs and/or restricted classes of protocols (e.g., stateless ones). A promising approach is given by the *computationally complete symbolic attacker (CCSA)* model, formalized in the BC Logic, which aims at bridging and getting the best of the two worlds, obtaining cryptographic guarantees by symbolic protocol analysis. The BC Logic is supported by a recently developed interactive theorem prover, namely SQUIRREL, which enables machine-checked interactive security proofs, as opposed to automated ones, thus requiring expert knowledge both in the cryptographic space as well as on the reasoning side.**

In this paper, we introduce the CRYPTOVAMPIRE cryptographic protocol verifier, which for the first time fully automates proofs of trace properties in the BC Logic. The key technical contribution is a first-order formalization of protocol properties with tailored handling of subterm relations. As such, we overcome the burden of interactive proving in higher-order logic and automatically establish soundness of cryptographic protocols using only first-order reasoning. Our first-order encoding of cryptographic protocols is challenging for various reasons. On the theoretical side, we restrict full first-order logic with cryptographic axioms to ensure that, by losing the expressivity of the higher-order BC Logic, we do not lose soundness of cryptographic protocols in our first-order encoding. On the practical side, CRYPTOVAMPIRE integrates dedicated proof techniques using first-order saturation algorithms and heuristics, which all together enable leveraging the state-of-the-art VAMPIRE first-order automated theorem prover as the underlying proving engine of CRYPTOVAMPIRE. Our experimental results showcase the effectiveness of CRYPTOVAMPIRE as a standalone verifier as well as in terms of automation support for SQUIRREL.

***Index Terms*—Security Protocols, Formal Methods, Computational Security, Automated Theorem Proving**

1. Introduction

Cryptographic protocols are the software interfaces used by the components of our digital world to communicate securely with one another. Unfortunately, designing such protocols is notoriously difficult and error-prone. From the classic attack on the Needham-Schroeder protocol [1], over to the ubiquitous but repeatedly broken TLS protocol [2], up to new and subtle blockchain protocols [3], the list of attacks on popular standards is ever-growing.

Formal methods have proved to be a very successful tool to guarantee properties of protocols and recently accompanied the design of protocol standards like TLS 1.3 [4], WireGuard [5], or 5G-AKA [6].

1.1. Related Work

Security properties for cryptographic protocols are typically formalized in terms of trace properties [7] or observational equivalence relations [8]. The former express guarantees on a single execution trace (possibly reflecting multiple protocol sessions) and cover secrecy of random data, authentication, and more general constraints on the order of security-relevant events. The latter express the inability of an attacker to tell the difference between two protocol configurations, such as the inability to understand which among two low-entropy secrets is used.

Formal verification of cryptographic protocols is further split into techniques working in the *symbolic* or *computational* model. Symbolic techniques typically abstract away from the algebraic properties of cryptography by reasoning over the so-called "Dolev-Yao" attacker [9], which has infinite computational resources at its disposal but is restricted to building only symbolic terms based on those already in their knowledge. This results in easier proof techniques, which enabled the design of several successful automated protocol verifiers like PROVERIF [10] and TAMARIN [11]. However, such a symbolic setting does not provide any computational guarantees: for instance, symbolically-secure protocols may, in fact, be attacked by leveraging weaknesses in the underlying cryptographic realization [12].

The computational cryptographic model is instead based on computational and probabilistic notions, which faithfully describe real-world implementations. In a computational